

Guia de Revisão para a Prova AAI

Machine Learning & Chatbot

1. Pré-processamento e Limpeza de Texto (NLP)

Um modelo de Machine Learning não entende "linguagem" da forma como nós entendemos - ele entende **padrões estatísticos em dados estruturados**. O problema é que a linguagem humana é cheia de ruído para fins estatísticos: maiúsculas e minúsculas que não mudam o significado, pontuação, palavras de preenchimento, variações de uma mesma palavra ("comprei", "comprando", "comprado"). Antes de qualquer modelo entrar em cena, existe uma etapa de **engenharia de limpeza**, cujo objetivo é reduzir essa variação sem perder o significado da mensagem.

Essa etapa é composta por três frentes principais:

Técnica	O que faz	Por que importa
Tokenização	Quebra o texto em unidades menores (<i>tokens</i>) - normalmente palavras e sinais de pontuação	É o primeiro corte: sem tokens, não há o que processar depois
Remoção de Stopwords	Elimina palavras de altíssima frequência e baixo valor discriminativo (artigos, preposições, conjunções)	Essas palavras aparecem em praticamente qualquer frase, então não ajudam o modelo a diferenciar uma intenção da outra
Stemming vs. Lematização	Reduz palavras flexionadas a uma forma comum	Evita que o modelo trate "comprar", "comprando" e "comprado" como três conceitos diferentes

O ponto que mais confunde na prova é a diferença entre **Stemming** e **Lematização** - e ela é simples se você pensar assim: **um corta, o outro consulta**.

Stemming é um corte mecânico de sufixo, baseado em regras fixas, sem entender gramática. É rápido, mas pode gerar palavras que não existem no dicionário (ex.: "gatinhos" → "gatinh"). Pense nele como uma tesoura: corta sem saber exatamente o que está cortando.

Lematização analisa a palavra dentro do seu contexto gramatical (é um verbo? um substantivo? está no plural?) e devolve a forma **canônica de dicionário** - o *lema*. É mais lenta, mas sempre entrega uma palavra real. Pense nela como um dicionário: você entra com a palavra flexionada e sai com a entrada oficial do verbete.

Para a prova: se a questão pedir "corte de sufixo, sem garantia de ser uma palavra válida" → **Stemming**. Se pedir "forma canônica reconhecida no dicionário" → **Lematização**.

Exemplo prático

Imagine que um chatbot de e-commerce recebe a seguinte mensagem de um cliente:

"Os meus produtos COMPRADOS não chegaram!"

A esteira de pré-processamento trata essa frase em quatro passos sequenciais:

Passo 1 - Tokenização. A frase é quebrada em unidades individuais, incluindo a pontuação:

["Os", "meus", "produtos", "COMPRADOS", "não", "chegaram", "!", ""]

Passo 2 - Normalização de caixa e remoção de ruído. Tudo vira minúsculo e a pontuação é descartada:

["os", "meus", "produtos", "comprados", "não", "chegaram"]

Passo 3 - Remoção de stopwords. "Os" e "meus" são artigos/pronomes de altíssima frequência - carregam pouquíssima informação sobre a *intenção* da mensagem, então saem:

["produtos", "comprados", "não", "chegaram"]

Passo 4 - Lematização. Cada palavra flexionada volta à sua forma de dicionário:

- "produtos" → "produto" (singular)
- "comprados" → "comprar" (particípio → infinitivo)
- "chegaram" → "chegar" (conjugado → infinitivo)

Resultado final, pronto para virar vetor numérico:

["produto", "comprar", "não", "chegar"]

Note que, mesmo perdendo a estrutura gramatical original, a frase resultante ainda comunica claramente o essencial: *produto comprado que não chegou* - exatamente a informação que o modelo precisa para classificar a intenção como um problema de entrega.

2. Vetorização de Texto: TF-IDF vs. Word Embeddings

Depois de limpo, o texto ainda é apenas uma lista de palavras - e modelos matemáticos não operam sobre palavras, e sim sobre números. A **vetorização** é a ponte entre linguagem e matemática: ela converte cada token (ou cada frase) em um vetor numérico. Existem duas filosofias bem diferentes para fazer essa conversão, e a prova costuma cobrar exatamente a diferença conceitual entre elas.

TF-IDF - importância estatística dentro de um corpus

TF-IDF significa *Term Frequency – Inverse Document Frequency*. A lógica é puramente estatística: uma palavra é **importante** se ela aparece muito em *um* documento específico, mas **pouco** no restante do corpus. Palavras raras e específicas recebem peso alto; palavras genéricas e onipresentes recebem peso baixo - mesmo que se repitam bastante.

Word Embeddings - significado como posição no espaço

Embeddings seguem uma lógica completamente diferente: em vez de contar frequências, eles representam cada palavra como um **vetor denso em um espaço contínuo**, aprendido a partir do contexto em que a palavra costuma aparecer. O resultado é que palavras com significados parecidos acabam **geometricamente próximas** nesse espaço - o modelo "entende" relações de sinonímia e associação sem que ninguém tenha programado essa regra manualmente.

Aspecto	TF-IDF	Word Embeddings
O que mede	Frequência ponderada da palavra no corpus	Relação semântica entre palavras
Tipo de vetor	Esparso (muitos zeros)	Denso (poucas dimensões, todas preenchidas)
Entende sinônimos?	Não	Sim
Base do cálculo	Estatística de ocorrência	Contexto de uso aprendido

Exemplo prático

Imagine uma imobiliária que usa IA para analisar mensagens de clientes e classificar chamados.

No TF-IDF: a palavra *"apartamento"* aparece em praticamente toda mensagem recebida - não importa se é uma dúvida sobre contrato, uma reclamação ou um elogio. Por ser tão frequente e tão pouco discriminativa, o TF-IDF atribui a ela um **peso baixo**. Já a palavra *"vazamento"* aparece raramente, e quase sempre associada a chamados urgentes de manutenção - por isso recebe um **peso alto**: estatisticamente, é uma palavra que sinaliza algo específico e relevante.

No Word Embedding: o modelo não está contando frequências, está posicionando palavras no espaço com base em como elas costumam ser usadas. Como *"aluguel"* e *"locação"* aparecem em contextos praticamente idênticos, seus vetores ficam quase sobrepostos - a distância entre eles é próxima de zero. Já *"aluguel"* e *"futebol"* nunca compartilham contexto, então a distância entre seus vetores é grande. É essa geometria que permite que o modelo reconheça que uma mensagem sobre *"locação"* deveria ser tratada como equivalente a uma mensagem sobre *"aluguel"*, mesmo usando palavras diferentes.

Para a prova: TF-IDF responde "essa palavra é rara e importante *neste corpus?*". Embeddings respondem "essa palavra *significa* algo parecido com outra?". São perguntas diferentes - por isso um modelo baseado só em TF-IDF nunca vai perceber que *"aluguel"* e *"locação"* são sinônimos.

3. Arquitetura de Chatbot NLU (Intenção, Entidade e Fallback)

Um chatbot de NLU (*Natural Language Understanding*) não "conversa" de forma livre - ele **traduz** a frase digitada pelo usuário em um comando estruturado que o sistema consegue executar. Essa tradução se apoia em três conceitos que sempre aparecem juntos:

- **Intenção (*Intent*):** é o *verbo* da conversa - o objetivo que o usuário quer alcançar (fazer um pix, cancelar um pedido, consultar um saldo).
- **Entidade (*Slot*):** são os *parâmetros* daquele objetivo - os dados específicos que o sistema precisa para executar a ação (quanto, para quem, quando).
- **Limiar de Confiança (*Threshold*) e Fallback:** é o mecanismo de segurança do sistema. O modelo nunca tem 100% de certeza - ele calcula uma probabilidade para cada intenção possível. Se a probabilidade da intenção mais provável ficar **abaixo** de um limiar definido (por exemplo, 70%), o sistema não arrisca executar a ação: aciona o **fallback**, que pode ser pedir para o usuário reformular a frase ou transferir o atendimento para um humano.

Pense nessa arquitetura como um **formulário sendo preenchido em tempo real por uma conversa**: a intenção decide qual formulário abrir, as entidades preenchem os campos, e o threshold decide se o sistema tem confiança suficiente para "enviar" esse formulário sozinho.

Exemplo prático

Um cliente escreve no chat de um banco:

"Quero transferir R\$ 150 para a Maria amanhã"

O motor de NLU processa a frase e devolve uma estrutura como esta:

- **Intenção identificada:** realizar_pix - confiança de **92%**
- **Entidades extraídas:**
 - valor → 150.00
 - destinatario → "Maria"
 - data_agendamento → "amanhã"

Decisão do sistema: como 92% está acima do limiar de 70% configurado, o sistema tem confiança suficiente para agir sozinho - a API executa a transação sem pedir confirmação extra.

Agora compare com uma mensagem ambígua do mesmo cliente:

"Paga lá o negócio pra ela"

Aqui não há verbo claro nem valores explícitos. O modelo ainda tenta associar a frase à intenção realizar_pix, mas com confiança muito menor - digamos, **40%**. Como esse valor está *abaixo* do limiar de 70%, o sistema **não executa nada automaticamente**. Em vez disso, aciona o fallback:

"Não entendi muito bem. Você pode me dizer o valor e para quem deseja transferir?"

Esse mecanismo é o que evita que um chatbot bancário execute uma transação financeira baseado em um "chute" de baixa confiança - o threshold funciona como uma rede de segurança entre a IA e a ação real no sistema.

4. Métricas de Avaliação de Classificação

Depois que um modelo de classificação está treinado, ainda falta responder a pergunta mais importante: **ele é confiável?** Para isso, comparamos as previsões do modelo com a realidade, e organizamos essa comparação em uma **Matriz de Confusão** - uma tabela que separa os acertos e os erros em quatro categorias:

	Previsto: Positivo	Previsto: Negativo
Real: Positivo	Verdadeiro Positivo (VP)	Falso Negativo (FN)
Real: Negativo	Falso Positivo (FP)	Verdadeiro Negativo (VN)

- **VP (Verdadeiro Positivo):** o modelo previu "sim", e era mesmo "sim".
- **VN (Verdadeiro Negativo):** o modelo previu "não", e era mesmo "não".
- **FP (Falso Positivo / alarme falso):** o modelo previu "sim", mas era "não" - o modelo se enganou disparando um alerta desnecessário.

- **FN (Falso Negativo / passou despercebido):** o modelo previu "não", mas era "sim" - o modelo deixou passar um caso real, sem perceber.

A partir dessa matriz, calculamos duas métricas que respondem a perguntas diferentes - e essa diferença é o ponto mais cobrado em prova:

Precisão responde: *"das vezes que o modelo gritou 'positivo', quantas vezes ele estava certo?"* Ou seja, seu foco é **evitar Falsos Positivos**.

Recall (Sensibilidade) responde: *"de todos os casos positivos que realmente existiam, quantos o modelo conseguiu encontrar?"* Ou seja, seu foco é **evitar Falsos Negativos**.

Para a prova: memorize assim - **Precisão olha para trás** (dos alertas que eu disparei, quantos eram reais?). **Recall olha para frente** (dos casos reais que existiam, quantos eu consegui pegar?).

Exemplo prático

Um modelo de IA é treinado para detectar **transações fraudulentas** em um e-commerce.

- **Cenário de Falso Positivo (FP):** o modelo bloqueia o cartão de um cliente honesto, achando (erradamente) que é fraude. Consequência: o cliente passa vergonha no caixa, fica irritado com a loja, mas nenhum prejuízo financeiro real ocorre.
- **Cenário de Falso Negativo (FN):** uma fraude real acontece e o modelo deixa passar, classificando a compra como legítima. Consequência: o e-commerce arca com o prejuízo financeiro direto da fraude.

Colocando os dois erros lado a lado, fica claro que eles não têm o mesmo custo para o negócio: um gera constrangimento, o outro gera prejuízo financeiro direto.

Qual métrica priorizar nesse cenário? O **Recall** deve ser priorizado. Em detecção de fraude, é preferível gerar alguns alarmes falsos a mais (FP) - que geram apenas incômodo e podem ser revertidos com uma simples confirmação - do que deixar fraudes reais passarem despercebidas (FN), que geram prejuízo direto e, em geral, irreversível.